

Language Design, DSLs, and Industry Case Studies

Liting Zhang

University of West Florida

Overview

- Language design trade-offs in the real world (constraints, stakeholders, risk)
- DSLs in industry: why they exist and why they fail
- DSL product thinking: scope, non-goals, safety, and versioning
- Using what we already covered (lexing/parsing/AST) as a black box
 - ▶ focus today: semantics, usability, tooling, governance
- Case studies: Go vs Java vs Rust vs TypeScript (decision framing)
- Why JS $\rightarrow$ TS worked (gradual typing + tooling incentives)
- JVM/CLR ecosystems: how runtimes + libraries shape language choices
- Industry angle: hiring, tooling, performance, maintainability, migration cost

Learning goals

After this lecture, you should be able to

- Evaluate language design decisions using constraints and trade-offs
- Decide when to use a DSL vs a general-purpose language vs config conventions
- Design a DSL scope: features, non-goals, and safety boundaries
- Specify DSL semantics: evaluation model, determinism, and error behavior
- Plan for adoption: tooling needs, testing strategy, and versioning
- Compare Go/Java/Rust/TypeScript decisions using concrete criteria
- Explain why gradual typing enabled JavaScript → TypeScript migration
- Describe how ecosystem and runtime affect long-term maintainability

Language design in practice

- 1 Language design in practice
- 2 DSLs: when and why
- 3 DSL productization
- 4 Implementing a DSL
- 5 Industry case studies
- 6 Capstone tie-in
- 7 Review

Language decisions are product decisions

In industry, language decisions are rarely academic.

- You are optimizing for an organization, not just a compiler
- Stakeholders include: engineers, security, SRE, PM, compliance, hiring
- Most “language design” happens in APIs, DSLs, and conventions

A helpful reframing:

- **What mistakes do we want to make impossible?**
- **What do we want to make easy and frequent?**
- **What do we want to make hard or forbidden?**

Constraint checklist engineers actually use

Typical constraints:

- Runtime constraints: latency (p99), throughput, memory footprint
- Safety constraints: untrusted input, sandboxing, compliance, auditability
- Operational constraints: observability, debugging, incident response
- Delivery constraints: CI/CD, build time, deployment model
- Team constraints: hiring pool, onboarding time, code review norms
- Migration constraints: existing codebase, interop, incremental rollout

Language features are only valuable if they reduce risk or cost under these constraints.

Trade-offs: a practical table mindset

A useful evaluation lens:

- Readability: can reviewers understand intent quickly
- Writability: can authors express common tasks with low friction
- Reliability: does the system prevent/contain mistakes
- Cost: training, tooling, runtime cost, maintenance cost

Design implies choosing defaults:

- defaults become culture (lint rules, style guides, idioms)
- defaults also become failure modes

A common pattern: constrain the dangerous parts

Organizations often prefer:

- a small set of safe primitives
- explicit escape hatches (rare, reviewed)
- tooling that catches mistakes early

Examples:

- restricted query languages instead of embedding arbitrary code
- typed APIs to prevent misuse
- safe-by-default concurrency patterns

Practice 1: identify constraints and pick a direction

Scenario: A company wants a policy system for access control.

- Policies must be auditable and reviewed
- Must not allow arbitrary code execution
- Must be fast at runtime and easy to test
- Policies change weekly; rollback is common

Task:

- List the top 3 constraints
- Would you prefer: (a) config, (b) DSL, or (c) general-purpose scripting
- Give one trade-off you accept

Practice 1 Answer: identify constraints and pick a direction

Likely top constraints:

- auditability and reviewability (humans must understand diffs)
- safety (no arbitrary code execution)
- testability and predictable runtime behavior

Likely choice:

- DSL or a constrained policy language
- config alone often becomes too limited (conditionals, composition)
- scripting is too powerful and risky for this domain

Trade-off accepted:

- upfront cost to build tooling and validation

DSLs: when and why

- 1 Language design in practice
- 2 DSLs: when and why
- 3 DSL productization
- 4 Implementing a DSL
- 5 Industry case studies
- 6 Capstone tie-in
- 7 Review

DSL definition (short)

Domain-Specific Language (DSL):

- A language for a narrow domain with constrained expressiveness
- Goal: make the common case easy and the dangerous case hard

Two forms:

- External DSL: its own syntax (SQL, regex, policy languages)
- Embedded DSL: fluent API inside a host language

Today: we care more about **semantics** + **adoption** than syntax mechanics.

Decision tree: config vs DSL vs code

A useful heuristic:

- Use **config** when:
 - ▶ mostly data, few rules, simple validation
- Use a **DSL** when:
 - ▶ rules/queries/policies need structure and safety
 - ▶ non-engineers edit it or diffs must be reviewed like contracts
- Use **code** when:
 - ▶ you truly need general computation and reuse
 - ▶ engineers own it and sandboxing is not required

Common DSL success patterns

DSLs succeed when they:

- have a stable, well-defined domain model
- benefit from static validation (types, schemas, constraints)
- benefit from optimization or caching (queries, rules)
- are authored by people who are not runtime engineers
- need audit trails and explicit semantics

Common DSL failure patterns

DSLs fail when:

- error messages are poor (users cannot debug)
- versioning is neglected (breaking changes accumulate)
- feature creep turns it into a weak general-purpose language
- tooling is missing (no formatter, no lint, no tests)
- security boundaries are unclear (untrusted inputs, injection)

A DSL without tooling is often worse than a small library API.

External DSL example: policy rules (readable diffs)

```
1 policy "admin_access" when user.role == "admin" and  
  ↪ resource.sensitivity <= 2 {  
2   allow;  
3 }  
4  
5 policy "readonly_guest" when user.role == "guest" {  
6   allow read;  
7   deny write;  
8 }
```

Benefits:

- reviewers can read intent
- constrained operators reduce surprises

Embedded DSL example: builder pattern (host tooling)

Embedded DSL style (TypeScript):

```
1 rule("free_shipping")
2   .when(and(eq(varOf("tier"), str("gold")), gte(varOf("total"),
3     ↪   num(50))))
4   .do(setVar("shipping", num(0)))
5   .do(emit("free shipping applied"));
```

Benefits and costs:

- gains TypeScript tooling and types
- loses plain-text readability for non-engineers

Practice 2: choose external vs embedded DSL

You need a discount rule system:

- marketing authors rules
- engineers review and deploy weekly
- you must guarantee no network/file access from rules

Task:

- choose external DSL or embedded DSL
- list two advantages and two risks of your choice

Practice 2 Answer: choose external vs embedded DSL

External DSL is usually favored here.

Advantages:

- easier to sandbox: no host-language arbitrary execution
- reviewable diffs for non-engineers, can enforce restricted constructs

Risks:

- you must build (and maintain) parsing + error messages + tooling
- feature pressure can expand scope rapidly

DSL productization

- 1 Language design in practice
- 2 DSLs: when and why
- 3 DSL productization**
- 4 Implementing a DSL
- 5 Industry case studies
- 6 Capstone tie-in
- 7 Review

A DSL is a contract, not just syntax

A usable DSL needs a spec:

- Syntax: what programs look like
- Semantics: what programs mean (evaluation order, types, effects)
- Errors: what happens on invalid input and how it is reported
- Compatibility: how changes are rolled out over time

Teams adopt DSLs when the contract is stable and debuggable.

Semantics decisions that matter

Questions your DSL must answer:

- Determinism: are results stable across runs
- Evaluation order: do rules run top-to-bottom, all-match, first-match
- Effects: can a rule update state that later rules observe
- Types/coercions: do you allow implicit conversions (usually avoid)
- Error behavior: fail-fast, collect errors, partial results

These decisions shape correctness more than the grammar does.

Rule engine semantics: three common models

Common execution models:

- All-match:
 - ▶ evaluate every rule, apply all actions
- First-match:
 - ▶ stop after first matching rule (priority semantics)
- Phased:
 - ▶ compute derived facts, then apply actions (reduces order bugs)

Picking one is a design choice with real bug consequences.

Mini spec example: rule order is explicit

A tiny spec excerpt:

```
1 Rules execute top-to-bottom.  
2  
3 A rule condition reads from the current environment.  
4  
5 Actions apply immediately:  
6   - set updates the environment  
7   - emit appends to the output log  
8  
9 If a condition evaluates to non-boolean, that is a runtime  
  ↪ error.
```

This prevents ambiguity like “does set affect later rules?”

Safety boundaries: sandboxing and budgets

If DSL input is untrusted:

- no file/network access
- no host eval or dynamic imports
- bounded computation (no unbounded loops)
- resource budgets:
 - ▶ max program size, max rules, max actions, max recursion depth

Reliability boundary:

- fail closed for policy/security DSLs
- record audit logs of evaluation and decisions

Tooling that makes a DSL viable

Minimum viable tooling:

- helpful error messages with line/column and context
- tests: golden files (input $\rightarrow$ output/errors)
- formatting / consistent style (even a simple pretty-printer)
- docs: examples and common patterns

Nice-to-have:

- editor integration (syntax highlighting)
- static checks (unknown vars, unreachable rules, type errors)

Versioning: you will need it earlier than you think

DSLs evolve. Plan for:

- feature flags / opt-in syntax
- deprecation warnings before removal
- strict mode vs legacy mode
- migration tools (even simple search-and-rewrite scripts)

Organizational reality:

- you will have many DSL programs across repos
- changes must be rolled out safely

Practice 3: choose rule engine semantics

You are designing a fraud rule DSL.

- Rules may emit alerts
- Some rules assign a risk score
- Engineers often add “exceptions” rules later

Task:

- choose all-match, first-match, or phased execution
- explain one bug that your choice avoids

Practice 3 Answer: choose rule engine semantics

A common good choice: phased.

- Phase 1: compute derived facts / risk score contributions
- Phase 2: emit alerts based on final score / facts

Bug avoided:

- order-dependent behavior where an early set changes later rule meaning
- “exception rules” become easier to reason about with explicit phases

Implementing a DSL

- 1 Language design in practice
- 2 DSLs: when and why
- 3 DSL productization
- 4 Implementing a DSL**
- 5 Industry case studies
- 6 Capstone tie-in
- 7 Review

Implementation architecture (high level)

We assume the pipeline exists:

- source text $\rightarrow$ tokens $\rightarrow$ AST

Today we emphasize what comes next:

- semantic checks (unknown variables, type mismatch)
- evaluation model and outputs
- errors designed for DSL authors
- test harness and fixtures

TypeScript: AST shape (minimal, for semantics discussion)

```
1  type Expr =
2    | { tag: "num"; value: number }
3    | { tag: "str"; value: string }
4    | { tag: "var"; name: string }
5    | { tag: "bin"; op: string; left: Expr; right: Expr };
6
7  type Action =
8    | { tag: "set"; name: string; expr: Expr }
9    | { tag: "emit"; msg: string };
10
11 type Stmt =
12   | { tag: "let"; name: string; expr: Expr }
13   | { tag: "rule"; name: string; cond: Expr; actions: Action[]
14     ↪   };
15
16 type Program = { stmts: Stmt[] };
```

We use this to discuss checks + semantics, not parsing mechanics.

Semantic checks: catch mistakes before runtime

Examples of static checks:

- unknown variable names (typos)
- type mismatch in operators (string ; number)
- rule condition must be boolean
- unreachable rules (if conditions are contradictory)

Why this matters:

- DSL authors may not have a debugger
- production evaluation should be predictable and safe

TypeScript: type checking sketch (small)

Minimal type universe:

```
1  type Ty = "num" | "str" | "bool";
2
3  type Sig = {
4    vars: Map<string, Ty>;
5  };
6
7  function typeOf(e: Expr, sig: Sig): Ty {
8    switch (e.tag) {
9      case "num": return "num";
10     case "str": return "str";
11     case "var": {
12       const t = sig.vars.get(e.name);
13       if (!t) throw new Error("unknown variable: " + e.name);
14       return t;
15     }
16     case "bin": {
17       const a = typeOf(e.left, sig);
18       const b = typeOf(e.right, sig);
19       if (e.op === "and" || e.op === "or") {
```

Practice 4: static checks

Suppose the signature says:

- total: num
- country: str

Which expressions should be rejected?

- 1 | A) `total >= 50`
- 2 | B) `country >= "US"`
- 3 | C) `total and (50 >= 10)`
- 4 | D) `country == "US"`

Practice 4 Answer: static checks

Accepted:

- A) `total i= 50 (num i= num) → bool`
- D) `country == "US" (str == str) → bool`

Rejected:

- B) `country i= "US"` (string ordering not supported in this DSL spec)
- C) `total and ...` (and expects bool operands; total is num)

Industry case studies

- 1 Language design in practice
- 2 DSLs: when and why
- 3 DSL productization
- 4 Implementing a DSL
- 5 Industry case studies**
- 6 Capstone tie-in
- 7 Review

How companies pick languages (a realistic rubric)

A rubric that maps to budgets:

- Build/CI time and developer iteration speed
- Operational cost: debugging, profiling, incident response
- Reliability: safety properties and failure modes
- Ecosystem: libraries, frameworks, hiring pool
- Deployment: artifacts, runtime footprint, cross-platform needs
- Migration cost: interop and incremental adoption

Most “debates” become clear when you name the constraints.

Go vs Java vs Rust vs TypeScript (one slide summary)

Typical strengths:

- Go: simplicity, tooling, deployment, concurrency ergonomics
- Java: ecosystem, mature runtime, observability tooling, portability
- Rust: memory safety without GC, performance, correctness under concurrency
- TypeScript: ecosystem reach (JS), IDE support, gradual adoption, API modeling

Typical risks:

- Go: less expressive typing, GC trade-offs for some latency profiles
- Java: footprint/warmup, complexity via large frameworks if uncontrolled
- Rust: learning curve, slower early velocity without mentorship
- TypeScript: types erased at runtime, complexity if type design is undisciplined

Why JS → TS worked (design lesson)

TypeScript's key design move:

- **gradual typing** and **structural typing** that matches JS reality
- adoption can be incremental (file-by-file, any/unknown escape hatches)
- huge payoff through tooling: refactoring, navigation, autocomplete

Language design lesson:

- migration paths can matter more than theoretical elegance

JVM/CLR ecosystems (design lesson)

VM ecosystems change the calculus:

- shared runtime services: GC, JIT, profiling hooks
- mature libraries and frameworks reduce time-to-market
- interoperability within the ecosystem can beat language purity

Lesson:

- language choice is often an ecosystem choice

Practice 5: choose a language with explicit constraints

Scenario A: internal developer CLI.

- quick iteration, good parsing libraries
- startup time matters a bit
- deployment must be simple across OSes

Scenario B: low-latency edge proxy.

- p99 latency is critical
- memory safety bugs are unacceptable
- team can invest in training

Task:

- pick a likely language for A and B
- justify by naming the top constraints you used

Practice 5 Answer: choose a language with explicit constraints

Scenario A:

- TypeScript/Node or Python: fast iteration + libraries
- Go: strong option if you want single binaries and consistent deployment

Scenario B:

- Rust: memory safety + predictable performance without GC pauses
- Go/Java can work depending on latency tolerance and operational maturity

Key point:

- your justification should map features to constraints, not preferences

Capstone tie-in

- 1 Language design in practice
- 2 DSLs: when and why
- 3 DSL productization
- 4 Implementing a DSL
- 5 Industry case studies
- 6 Capstone tie-in**
- 7 Review

Capstone expectation (aligned with prior weeks)

Your capstone DSL should emphasize:

- clear domain story (who uses it, what decisions it controls)
- crisp scope (features) and non-goals (what it cannot do)
- semantics: evaluation model and error behavior
- validation: static checks where possible
- tests: golden inputs and expected outputs/errors

Parsing and AST are necessary, but the differentiator is: **usability and correctness**.

A good “non-goals” slide (saves projects)

Example non-goals you can explicitly write:

- no loops, recursion, or unbounded computation
- no file/network access, no host-language eval
- no user-defined functions (unless your domain truly needs them)
- no implicit type coercions

These reduce feature creep and clarify your safety boundary.

Practice 6: write a DSL spec snippet

Pick a domain (one):

- alerting rules
- access-control policies
- feature flag targeting

Task:

- Write 3 semantics bullets (order, determinism, error handling)
- Write 2 non-goals

Practice 6 Answer: example (feature flags)

Semantics bullets:

- Rules execute top-to-bottom; first-match stops evaluation
- Conditions are pure: no time/random/network; deterministic for same inputs
- On error (unknown var/type mismatch), evaluation fails closed and returns an error report

Non-goals:

- No loops or user-defined functions
- No side effects beyond returning a decision and an explanation log

Review

- 1 Language design in practice
- 2 DSLs: when and why
- 3 DSL productization
- 4 Implementing a DSL
- 5 Industry case studies
- 6 Capstone tie-in
- 7 Review**

Summary: what to remember

- Language design in industry is constraint-driven decision-making
- DSLs succeed when scope is narrow and semantics are explicit
- The hard parts are usability: errors, tooling, tests, versioning, governance
- “Config vs DSL vs code” is a recurring architecture choice
- Migration paths (like JS $\rightarrow$ TS) can matter more than purity
- Ecosystems (JVM/CLR, JS) strongly influence long-term cost and hiring
- Capstone DSLs should prioritize semantics + validation + tests over feature count

Review questions

- Name three constraints that dominate language decisions in industry
- When is a DSL a better fit than config or code
- What is one DSL failure mode and how do you prevent it
- Why does versioning matter even for “small” DSLs
- What design choice made TypeScript adoption feasible for JS codebases
- How do JVM/CLR ecosystems change language selection trade-offs